Project 3: Parallel image augmentation

Tommaso Ciccotti
7196985
tommaso.ciccotti@edu.unifi.it

Abstract

This project compares deterministic image augmentations using Albumentations and Kornia on CPU and CUDA. Sixteen resolution–batch pairs over four resolutions are combined with seven augmentation profiles producing 112 workloads. Each workload evaluates one Albumentations sequential configuration, six Albumentations ThreadPool configurations, six Kornia CPU configurations and two CUDA timing scopes collected from the same CUDA execution for a total of 1568 configurations. Albumentations is the fastest CPU library in all workloads, its ThreadPool path is the best in 87 cases and reaches a maximum internal speedup of $5.47\times$*. CUDA device-only timing is faster than the best CPU result in 61 workloads and reaches* $10.76\times$ *while the complete end-to-end CUDA is faster in 12 workloads and reaches a* $2.93\times$ *speedup. The crossover depends on resolution, batch size and transformation type because the GPU computation must amortize the launch overhead and the host–device transfers. Different augmentation profiles are better suited for one implementation or the other based on the size of the workload they provide. All configurations satisfy the output tolerance and the minimum SSIM is* 0.996780*.*

1. Introduction

Image augmentation generates modified versions of source images through geometric, colour and filtering operations. It is commonly applied before a computer-vision or machine-learning model receives the data. The augmentation stage becomes computationally relevant when image size, batch size or the number of transformations increases.

The project evaluates two Python libraries. Albumentations operates on NumPy images and uses OpenCV for the main image operations. Kornia applies the transformations to PyTorch tensors and executes the same pipeline on the CPU or on CUDA.

The Albumentations implementation is tested sequentially and with a CPU thread pool. Every image in a batch is independent so different images can be assigned to different workers, a batch is a group of B images processed in one benchmark call. Albumentations receives it as a list of independent RGB images while Kornia stores it as one tensor with shape $B \times C \times H \times W$, where $C = 3$ for RGB data.

Each Kornia CUDA repetition produces two timings from one execution. A host timer measures the complete transfer–augmentation–transfer path while CUDA events delimit the augmentation interval on the device.

All transformations use deterministic parameters. The same sample receives the same flip, angle, translation, scale, brightness, contrast and blur values in every backend and repetition. The output is verified with exact equality and numerical image metrics.

2. Implementation

The application loads one image, a directory of images or a deterministic synthetic RGB image. The source images are resized to the requested square resolution and repeated until the batch is complete. The benchmark then executes the configured profiles with Albumentations and Kornia.

2.1. Code organization

The project separates transformation execution, metric calculation, CSV reporting and workload orchestration into dedicated modules.

Table 1. Responsibilities of the benchmark modules. *Legend:* Each module contains one principal responsibility of the benchmark.

File	Responsibility
`backends.py`	Albumentations and Kornia preparation, execution and image–tensor conversion
`metrics.py`	Host and PyTorch timing, descriptive statistics and output-comparison metrics
`report.py`	CSV schema, row construction, numeric formatting and per-row flushing
`benchmark.py`	Workload iteration, backend orchestration, best-CPU selection and CUDA timing scopes
`config.py`	Profiles, deterministic sample parameters, paths and workload plan
`app.py`	Interactive menu, image loading, batch construction, preview and environment report

2.2. Deterministic parameters

Every augmentation profile defines maximum transformation values. The sign and magnitude applied to sample i follow a fixed cycle. Alternating signs change the rotation, translation, brightness and contrast direction while the magnitude cycle uses the values 1.00, 0.65, 0.35 and 0.85.

For an affine transformation the image centre is (c_x, c_y). Rotation angle θ and scale s define

$$\alpha = s\cos(\theta), \qquad \beta = s\sin(\theta). \tag{1}$$

The matrix used for rotation, scale and translation is

$$A = \begin{bmatrix} \alpha & \beta & (1-\alpha)c_x - \beta c_y + t_x \\ -\beta & \alpha & \beta c_x + (1-\alpha)c_y + t_y \end{bmatrix}. \tag{2}$$

Both paths use bilinear interpolation and reflected borders.

Brightness and contrast are represented as

$$I_{out} = \mathrm{clip}(cI_{in} + b, 0, 1), \tag{3}$$

where c is the contrast factor and b is the brightness offset.

Gaussian blur applies the same kernel size and standard deviation in both directions. The one-dimensional weight at offset k is

$$G(k, \sigma) = \exp\left(-\frac{k^2}{2\sigma^2}\right). \tag{4}$$

2.3. Albumentations CPU

A fixed Albumentations transformation is prepared for every image before timing begins. The sequential implementation applies the prepared transformations with a normal loop and preserves the input order.

The parallel implementation uses `ThreadPoolExecutor`. One task contains one image and its corresponding fixed transformation. The executor is created before the warm-ups and reused during all measured repetitions, pool creation is therefore excluded from the reported time.

OpenCV internal threading is fixed to one, this prevents the sequential path from using hidden parallelism and prevents each ThreadPool worker from starting additional OpenCV worker groups. The tested worker counts are 1, 2, 4, 6, 8 and 12.

The sequential output is Albumentations correctness reference. A threaded configuration can become the best CPU result only when it is exactly equal to the sequential output and has a lower mean time.

2.4. Kornia CPU

The input batch is converted from RGB uint8 NumPy arrays to a contiguous BCHW float tensor in the interval $[0, 1]$. The profile parameters are stored in tensors with one value per sample. The final tensor is clamped to the valid interval and converted back to RGB uint8 images for verification.

The tensor conversion is performed before timing and the same Kornia CPU operation is then measured with 1, 2, 4, 6, 8 and 12 PyTorch threads. The inter-operation thread count remains fixed to one so only the internal parallelism of each tensor operation changes. The one-thread Kornia time is the reference used to calculate Kornia CPU speedup and efficiency.

2.5. Kornia CUDA

Kornia CUDA uses the same transformation parameters and operations sequence used by Kornia CPU. The input tensor is copied to the GPU,

augmented and copied back to the CPU once per warm-up and measured repetition.

A host timer surrounds the complete execution:

$$T_{E2E} = T_{H2D} + T_{aug} + T_{D2H} + T_{sync}. \quad (5)$$

The final synchronization guarantees that the host sample includes completed GPU work.

Two CUDA events are recorded in the same stream immediately before and after the Kornia augmentation:

$$T_{device} = T_{event,end} - T_{event,start}. \quad (6)$$

Peak allocated CUDA memory is measured in the complete execution.

2.6. Augmentation profiles

Seven profiles separate framework overhead from geometric, colour and filtering work.

Table 2. Augmentation profiles used by the benchmark.

Profile	Operations
Identity	No transformation
GeometricLight	Horizontal flip, 8° rotation, translation and scale
GeometricStrong	Two flips, 18° rotation, translation and scale
Color	Brightness and contrast
Blur	Gaussian blur with kernel 7
MixedLight	Light affine, colour and Gaussian blur
MixedStrong	Strong affine, colour and Gaussian blur

Identity isolates framework and scheduling overhead. Geometric profiles emphasize interpolation while color mainly performs element-wise operations. Blur increases neighbourhood accesses and memory traffic while mixed profiles execute the complete transformation chain.

3. Setup

3.1. Hardware

The tests were executed on a desktop system with an **AMD Ryzen 5 3600X**. The processor contains six physical cores and can support up to twelve logical threads through simultaneous multithreading. The system contains 16 GB of DDR4 memory. CUDA execution uses an **NVIDIA GeForce RTX 2070 SUPER**.

3.2. Benchmark

A resolution–batch pair defines the input shape of one test and the sixteen pairs in Table 4 are evaluated with all the augmentation profiles. Batch one measures single-image latency while the larger batches increase mainly by powers of two so that scheduling, batching and transfer amortization can be observed without testing many nearly equivalent intermediate values.

Table 3. Execution environment.

Component	Value
CPU	AMD Ryzen 5 3600X, 6 cores / 12 threads
System memory	16 GB DDR4
GPU	NVIDIA GeForce RTX 2070 SUPER
Input	Deterministic synthetic RGB image
Warm-ups	2 per configuration
Repetitions	5 per configuration

Table 4. Benchmark workload matrix and maximum measured memory load.

Legend: Max Mpix is resolution area multiplied by the largest batch; MixedStrong CUDA; Peak memory in MB.

Res.	Batch sizes	Max Mpix	Peak MB
512^2	1, 4, 8, 16, 32, 64	16.8	1164.1
1024^2	1, 4, 8, 16, 32	33.6	2316.1
2048^2	1, 4, 8	33.6	2314.1
4096^2	1, 2	33.6	2312.9

The upper batch limits decrease as resolution grows. The largest 1024^2, 2048^2 and 4096^2 workloads each contain approximately 33.6 million input pixels and reach a similar MixedStrong CUDA memory peak of about 2.31 GB. At 512^2 batch 64 already exposes the small-image saturation region while avoiding an additional batch 128 test that would add many images without introducing a new execution regime.

The 4096^2 limit is due to tensor storage. One RGB `float32` tensor at this resolution requires approximately 192 MiB per image. Batch two therefore requires about 384 MiB for one tensor and MixedStrong reaches 2312.9 MB so larger batches were excluded to avoid excessive GPU allocation and system-memory pressure.

For each resolution–batch pair the seven profiles produce

$$16 \cdot 7 = 112 \quad (7)$$

workloads. Every workload becomes 14 measured configurations:

$$1 + 6 + 6 + 1 = 14. \quad (8)$$

These are composed by one Albumentations sequential, six Albumentations ThreadPool, six Ko-

rnia CPU and one CUDA configurations. The complete experiment therefore contains

$$112 \cdot 14 = 1568 \tag{9}$$

configurations. For each of them two warm-ups followed by five measured repetitions are done.

3.3. Timing and metrics

Execution time is measured after two warm-up runs and five repetitions are then recorded for every configuration. Their arithmetic mean is used as the principal performance value:

$$T_{\text{avg}} = \frac{1}{R} \sum_{i=1}^{R} T_i, \qquad R = 5. \tag{10}$$

The minimum and maximum times identify the fastest and slowest repetitions and help reveal isolated delays.

Timing variation is measured with the population standard deviation:

$$\sigma_T = \sqrt{\frac{1}{R} \sum_{i=1}^{R} (T_i - T_{\text{avg}})^2}. \tag{11}$$

Standard deviation is expressed in milliseconds and grows with the workload duration. The coefficient of variation removes this scale:

$$CV = \frac{\sigma_T}{T_{\text{avg}}} \times 100. \tag{12}$$

A low CV indicates that the repetitions remain close to the mean. Standard deviation and CV are analysed together because a long workload can have a larger absolute deviation but a smaller relative variation.

Within-library CPU speedup compares a parallel configuration with the corresponding one threaded path:

$$S_p = \frac{T_1}{T_p}. \tag{13}$$

The corresponding CPU parallel efficiency is

$$E_p = \frac{S_p}{p}. \tag{14}$$

Albumentations uses its sequential loop as the baseline while Kornia uses its one thread tensor execution. These values describe how each library scales internally. Albumentations and Kornia use different CPU parallelization strategies, therefore their efficiencies are not a perfectly symmetric comparison.

Direct backend comparisons instead use the fastest valid CPU result among Albumentations sequential, Albumentations ThreadPool and Kornia CPU:

$$T_{\text{bestCPU}} = \min\left(T_{\text{Alb,seq}}, T_{\text{Alb,pool}}, T_{\text{Kornia,CPU}}\right). \tag{15}$$

CUDA speedups are therefore

$$S_E = \frac{T_{\text{bestCPU}}}{T_{\text{E2E}}}, \qquad S_D = \frac{T_{\text{bestCPU}}}{T_{\text{device}}}. \tag{16}$$

This prevents a fast CUDA result from being compared only with a slower configuration from the same library.

Batch scaling is studied through time per image:

$$T_{\text{image}} = \frac{T_{\text{avg}}}{B}. \tag{17}$$

This value shows whether larger batches amortize scheduling, launch and transfer costs. Resolution scaling is analysed by comparing the same profile and batch across increasing image sizes.

The mean absolute error measures the average difference between corresponding 8-bit channel values:

$$MAE = \frac{1}{N} \sum_{i=1}^{N} |x_i - y_i|. \tag{18}$$

Structural similarity is measured with SSIM. A value of one represents identical images while values close to one indicate that luminance, contrast and image structure are preserved. Exact matches, MAE and minimum SSIM are used to distinguish identical executions from small interpolation differences without introducing redundant error metrics.

4. Results analysis

4.1. Output consistency

Consistency is evaluated against a reference selected according to the implementation. Albumentations ThreadPool and Kornia CPU are compared with Albumentations sequential to verify whether the two libraries preserve the same intended transformations. CUDA end-to-end is compared with the fastest valid CPU output. CUDA device-only is instead compared with the one thread Kornia CPU output.

Table 5. Output consistency by backend and reference.
Legend: Exact is the number of rows identical to the indicated reference.

Backend	Reference	Rows	Exact	Mean MAE	Min SSIM
Alb. sequential	Self	112	112	0.000000	1.000000
Alb. ThreadPool	Alb. sequential	672	672	0.000000	1.000000
Kornia CPU	Alb. sequential	672	132	0.419044	0.996781
CUDA E2E	Best CPU	112	22	0.419134	0.996780
CUDA device	Kornia CPU 1T	112	48	0.000106	0.999999

All configurations satisfy the tolerance rule. Albumentations ThreadPool is exactly equal to the sequential implementation in every workload, confirming that distributing independent images among workers does not modify the output.

Kornia CPU and CUDA end-to-end have a mean MAE close to 0.42 when compared with the Albumentations reference. Their minimum SSIM remains above 0.996 showing that the two libraries preserve the same image structure despite differences in interpolation, tensor arithmetic and rounding.

CUDA device-only mean MAE is only 0.000106 and its minimum SSIM is 0.999999. The Kornia CPU and CUDA implementations therefore produce almost identical results when the same Kornia pipeline is compared across devices. The larger errors mainly describe differences between Albumentations and Kornia while CPU and CUDA Kornia still show small numerical differences.

4.2. Backend comparison

MixedStrong is compared across every configured resolution and batch size. The CPU column is the fastest result among all thirteen CPU candidates and Albumentations provides the best CPU result in every case. End-to-end is the main CUDA measurement while device-only is reported as a secondary measurement .

Table 6. MixedStrong backend comparison.
Legend: Times are in ms; S_E and S_D are end-to-end and device-only CUDA speedups over the best CPU result.

Res.	B	CPU	E2E	Dev.	S_E	S_D
512^2	1	2.997	32.175	29.363	0.09	0.10
512^2	4	4.343	25.267	17.818	0.17	0.24
512^2	8	7.146	23.589	14.557	0.30	0.49
512^2	16	13.380	28.492	13.962	0.47	0.96
512^2	32	26.193	50.583	21.723	0.52	1.21
512^2	64	50.504	92.522	36.193	0.55	1.40
1024^2	1	15.393	40.569	32.198	0.38	0.48
1024^2	4	21.238	28.353	13.690	0.75	1.55
1024^2	8	31.672	50.846	21.639	0.62	1.46
1024^2	16	68.351	94.525	36.952	0.72	1.85
1024^2	32	129.777	180.755	67.775	0.72	1.91
2048^2	1	67.150	28.578	13.854	2.35	4.85
2048^2	4	89.955	93.002	36.680	0.97	2.45
2048^2	8	140.898	180.858	68.103	0.78	2.07
4096^2	1	269.633	92.083	35.473	2.93	7.60
4096^2	2	285.693	183.462	69.564	1.56	4.11

At 512^2 end-to-end CUDA remains slower than the best CPU result for every batch. The speedup rises from $0.09\times$ to $0.55\times$ because batching reduces the fixed cost per image but the complete CUDA path still does not recover its overhead. At 1024^2 the same behaviour remains: the best end-to-end result reaches only $0.75\times$.

The one batch measurements at 1024^2 and 2048^2 do not follow the expected resolution trend. The 1024^2 result is highly variable while the 2048^2 result is stable but lower than the 512^2 and 1024^2 times. These two points are therefore not used alone to infer resolution scaling. At 4096^2 CUDA end-to-end remains advantageous for both configured batches, reaching $2.93\times$ at batch one and $1.56\times$ at batch two as the workload allow to amortize the overhead.

Table 7. MixedStrong average time per image.
Legend: Bmax is the largest configured batch; all times are mean ms per image.

Res.	Bmax	CPU B1	CPU Bmax	E2E B1	E2E Bmax
512^2	64	2.997	0.789	32.175	1.446
1024^2	32	15.393	4.056	40.569	5.649
2048^2	8	67.150	17.612	28.578	22.607
4096^2	2	269.633	142.847	92.083	91.731

The time per image shows more clearly how batching changes the throughput. From batch one

to the largest batch the best CPU time per image falls by about 74% at 512^2, 1024^2 and 2048^2, and by 47% at 4096^2. CUDA end-to-end falls from 32.175 to 1.446 ms per image at 512^2 and from 40.569 to 5.649 ms at 1024^2. At 4096^2 E2E time per image is almost unchanged, from 92.083 to 91.731 ms, showing that one image already provides enough parallel work and that the second image mainly increases the amount of data processed.

Device-only follows the same general saturation trend. It becomes faster than the CPU earlier than end-to-end because transfers and host-side work are excluded. This difference is useful to estimate the potential of a pipeline that keeps the next processing stages on the GPU but it does not replace the complete E2E comparison.

4.3. CPU comparison and scaling

The largest configured batch of each resolution is used to compare the fastest measured Albumentations and Kornia CPU configurations under their respective parallelization strategies.

Table 8. CPU comparison for MixedStrong under the tested parallelization strategies.
Legend: T is the best worker or thread count; K/A is Kornia time divided by Albumentations time.

Res.	B	Alb. ms	Alb. T	Kornia ms	K. T	K/A
512^2	64	50.504	12	715.999	8	14.18
1024^2	32	129.777	12	1396.786	12	10.76
2048^2	8	140.898	8	1447.585	12	10.27
4096^2	2	285.693	2	1790.215	4	6.27

Albumentations is the fastest CPU library in all the 112 workloads. For the largest MixedStrong batches Kornia CPU is between $6.27\times$ and $14.18\times$ slower. The OpenCV-based Albumentations path benefits from efficient image kernels and from assigning independent images directly to the workers. Kornia instead applies a sequence of floating-point tensor operators whose intermediate tensors and memory traffic are more expensive on this CPU.

Albumentations uses twelve workers at 512^2 and 1024^2, eight at 2048^2 and two at 4096^2. The final case contains only two images so more than two image-level workers wouldn't be useful. Kornia uses eight, twelve, twelve and four threads because it parallelizes operations inside the tensor rather than assigning only one image to each thread.

4.4. CPU parallel efficiency

Parallel efficiency is evaluated inside each CPU library because the two paths expose different forms of parallel work.

Table 9. CPU parallel efficiency for the largest MixedStrong batches.
Legend: S is within-library speedup; B is the batch size; p indicates the thread count; $E = S/p$; values are percentages.

Res.	B	Alb. p	Alb. E	K. p	K. E	K/A
512^2	64	12	35.6%	8	22.0%	$14.18\times$
1024^2	32	12	32.4%	12	14.8%	$10.76\times$
2048^2	8	8	46.1%	12	16.1%	$10.27\times$
4096^2	2	2	84.2%	4	39.4%	$6.27\times$

Albumentations obtains the highest efficiency at 4096^2 because only two large images are processed and two workers can remain useful with little image-level oversubscription. At 512^2 and 1024^2 the efficiency is lower because twelve workers share smaller images and the scheduling overhead has a larger relative effect.

Kornia CPU efficiency is lower in the largest batches. Its threads cooperate inside tensor operations so additional threads also increase synchronization and memory-traffic pressure. The efficiency gap helps explain the CPU comparison.

4.5. Kornia CUDA timing scopes

End-to-end is the main CUDA benchmark because it measures the complete execution. Device-only is collected from the same MixedStrong execution as a secondary timing scope that isolates the augmentation after the tensor is already on the GPU.

The largest workloads, 1024^2 B32, 2048^2 B8 and 4096^2 B2 all contain 33,554,432 pixels. Their E2E means are 180.755, 180.858 and 183.462 ms and their extra times are 112.980, 112.755 and 113.899 ms. The similar values show that the complete time follows the total amount of image data more closely than the resolution of one image alone.

Table 10. CUDA timing composition across resolutions and batch sizes.
Legend: Extra is end-to-end minus device-only time; Share is the device percentage of end-to-end time.

Res.	B	E2E ms	Device ms	Extra ms	Share
512^2	1	32.175	29.363	2.811	91.3%
512^2	8	23.589	14.557	9.032	61.7%
512^2	64	92.522	36.193	56.329	39.1%
1024^2	1	40.569	32.198	8.371	79.4%
1024^2	8	50.846	21.639	29.207	42.6%
1024^2	32	180.755	67.775	112.980	37.5%
2048^2	1	28.578	13.854	14.724	48.5%
2048^2	4	93.002	36.680	56.322	39.4%
2048^2	8	180.858	68.103	112.755	37.7%
4096^2	1	92.083	35.473	56.610	38.5%
4096^2	2	183.462	69.564	113.899	37.9%

The 512^2 B64 workload contains half as many pixels and its extra time is 56.329 ms, almost half of the values above. This supports the near-linear relation between transferred tensor volume and the part of E2E time outside the device interval.

Device-only accounts for about 37.5%–39.1% of the E2E time in the largest configured workloads.

4.6. Timing stability

Standard deviation and CV are compared together because they describe absolute and relative variation.

Table 11. Timing stability by backend.
Legend: Mean std. is in ms; lower CV indicates more stable repetitions.

Backend	Mean std.	Median CV	Maximum CV
Alb. sequential	1.231	1.64%	18.85%
Alb. ThreadPool	1.123	3.31%	58.26%
Kornia CPU	3.361	1.14%	126.40%
CUDA E2E	5.536	3.09%	89.19%
CUDA device	3.485	3.43%	132.11%

Kornia CPU has the lowest median CV at 1.14% followed by Albumentations sequential at 1.64%. CUDA end-to-end has a median CV of 3.09% so its repetitions remain reasonably close to the mean even though some short configurations contain large outliers.

The maximum E2E CV is 89.19% for GeometricLight at 512^2 batch 8. MixedStrong at 1024^2 batch 1 also has a high CV of 55.67%. The 2048^2 batch-one result has a low CV but an unexpectedly low mean compared with the lower resolutions. Kornia CPU has one of the highest maximum CV but its mean is the lowest.

CUDA device-only has a slightly higher median CV of 3.43% and its maximum reaches 132.11% for GeometricLight at 2048^2 batch 1.

4.7. Minimum and maximum results

The extrema show the effect of workload size, parallel overhead and timing outliers.

Table 12. Global timing and speedup extrema.
Legend: B is batch size; T is the thread count.

Metric	Minimum	Maximum
Mean runtime	0.00806 ms Alb. seq., Identity 2048^2, B1	2822.4 ms K. CPU 1T, MixedStrong 4096^2, B2
Standard deviation	0.000417 ms Alb. seq., Identity 4096^2, B1	63.46 ms K. CPU 2T, Geom. strong 4096^2, B1
Timing CV	0.04% K. CPU 1T, Blur 2048^2, B8	132.11% CUDA device, Geom. light 2048^2, B1
Alb. speedup	$0.10\times$ Identity, 8T 1024^2, B4	$5.47\times$ Geom. strong, 12T 512^2, B64
Kornia CPU speedup	$0.31\times$ Geom. light, 8T 512^2, B1	$3.03\times$ Geom. light, 12T 512^2, B8
CUDA E2E speedup	$0.0001\times$ Identity 4096^2, B1	$2.93\times$ MixedStrong 4096^2, B1
CUDA device speedup	$0.0009\times$ Identity 2048^2, B1	$10.76\times$ Geom. light 4096^2, B1

Identity produces the minimum runtime because it performs almost no augmentation work. For the same reason it also produces the lowest speedups, thread management and backend overhead are larger than the useful computation so both CPU parallelism and CUDA can be slower than the sequential path.

Timing variability is generally limited but the extrema reveal a few short or irregular workloads. The minimum standard deviation is almost zero while the maximum reaches 63.46 ms in a long Kornia CPU geometric workload. The maximum CV of 132.11% occurs in CUDA device-only GeometricLight at 2048^2 batch one and reflects a large relative variation between repetitions rather than the typical stability of the backend.

Albumentations reaches a maximum internal speedup of $5.47\times$ while Kornia CPU reaches $3.03\times$. Both also contain configurations below $1\times$, showing that additional workers or threads are not beneficial when the available work is too

small. The strongest complete CUDA result is the $2.93\times$ end-to-end speedup of MixedStrong at 4096^2 batch one. Device-only reaches $10.76\times$ on GeometricLight at the same resolution showing the potential of the augmentation kernels when the data are already resident on the GPU.

4.8. Augmentation profile comparison

Batch one is used because it is the only batch shared by all four resolutions. Table 13 uses end-to-end CUDA as the main comparison with the fastest CPU result across both libraries.

Table 13. CUDA end-to-end speedup by profile at batch 1. *Legend:* Each value is CUDA end-to-end speedup over the best CPU result; values above one favour CUDA.

Profile	512^2	1024^2	2048^2	4096^2
Identity	0.005	0.001	0.000	0.000
Geom. light	0.098	0.275	1.147	2.394
Geom. strong	0.119	0.315	1.502	2.674
Color	0.015	0.043	0.252	0.324
Blur	0.045	0.076	0.461	0.576
Mixed light	0.076	0.378	2.220	2.466
Mixed strong	0.093	0.379	2.350	2.928

At 512^2 and 1024^2 with every profile the end-to-end is slower than the best CPU implementation. The augmentation work is not large enough to recover the complete CUDA overhead. Identity remains close to zero because the CPU performs almost no image processing and is therefore not a useful acceleration workload.

At 2048^2 the mixed profiles report mean E2E speedups of $2.22\times$ for MixedLight and $2.35\times$ for MixedStrong while Color and Blur remain below the CPU at $0.25\times$ and $0.46\times$. GeometricLight and GeometricStrong also have mean speedups above one but show high timing variability.

At 4096^2 the result is stable and profile dependent. GeometricLight and GeometricStrong reach $2.39\times$ and $2.67\times$ while MixedLight and MixedStrong reach $2.47\times$ and $2.93\times$. Color and Blur remain slower at $0.32\times$ and $0.58\times$. Their lower computational intensity relative to the complete transfer and framework costs prevents the device advantage from surviving the E2E overhead even at batch one.

Device-only gives a different view of the same profiles. At 4096^2 it reaches about $10.7\times$ for the geometric profiles, 7.2–$7.6\times$ for the mixed profiles and about $3\times$ for Color and Blur. These values show that the kernels themselves benefit from the GPU.

5. Conclusions

The benchmark compares Albumentations and Kornia under one deterministic workload matrix and the same transformation parameters. Output checks are applied to every backend.

Albumentations provides the fastest CPU result in all the 112 workloads. Its ThreadPool implementation is the best in 87 cases and the sequential path in 25. The maximum internal speedup is $5.47\times$ compared with $3.03\times$ for Kornia CPU. The CPU efficiency analysis confirms that the two implementations use parallelism differently: Albumentations distributes independent images among the workers while Kornia uses PyTorch intra-operation parallelism for the tensor operations. The remaining gap also depends on the OpenCV-based Albumentations kernels and on Kornia's floating-point tensor pipeline.

CUDA end-to-end is faster than the best CPU result in 12 of the 112 workloads and reaches a maximum speedup of $2.93\times$ with MixedStrong at 4096^2 batch one. At higher resolutions the geometric and mixed profiles provide enough work to recover the complete CUDA overhead while Color and Blur remain slower at batch one even at 4096^2.

Batching improves the throughput but its effect depends on the resolution. For MixedStrong the E2E time per image falls strongly at 512^2 and 1024^2 while at 4096^2 it changes by only about 0.4% between batch one and two. The CPU continues to benefit from batch-level parallelism even with larger batches which can reduce the relative CUDA E2E advantage as the batch grows.

CUDA device-only is faster than the best CPU result in 61 workloads and reaches a speed-up of $10.76\times$. In the largest MixedStrong workloads the device part represents only about 38% of the E2E time. All cases satisfy the output tolerance but Albumentations ThreadPool is exactly equal

to its sequential reference while Kornia CPU and CUDA show small numerical differences even when their outputs remain visually identical.

Overall Albumentations is the strongest CPU choice. Kornia CUDA becomes advantageous when the image resolution and the transformation cost are large enough to recover the complete overhead. Device-only timing remains useful for estimating a GPU resident pipeline.

6. Future work

- *Real datasets*: repeat the experiment with photographs containing different textures, shapes and aspect ratios.
- *Additional operations*: add noise, sharpening and random erasing with deterministic parameters in both libraries.
- *More batches and resolutions*: repeat the experiment with more resolutions and batches to further study the behaviour of the two libraries.
- *Changing the type of parallelism*: repeat the experiment with both intra and inter-operation thread configurations for both libraries.